AI-based estimation of embedded software execution cycles in host-compiled simulation

Raúl Gómez-Varela, David Aledo, Héctor Posadas, Eugenio Villar
Departamento TEISA, Universidad de Cantabria, Santander, Spain

2024 39th Conference on Design of Circuits and Integrated Systems (DCIS) | 979-8-3503-6439-2/24/$31.00 ©2024 IEEE | DOI: 10.1109/DCIS62603.2024.10769196

***Abstract*—In Embedded and Cyber-Physical Systems (E&CPS) the functional and temporal interactions between the digital and physical parts are crucial. In these systems, most of the functionality is implemented by software deployed on heterogeneous, distributed platforms so its performance largely affects the whole E&CPS behavior. Their HW/SW co-design requires fast and accurate simulation tools capable of evaluating the performance of each platform configuration for the applications selected while minimizing software porting needs. Host-compiled simulation avoids porting, but has limitations to obtain accurate results due to the difficulty of extracting and modeling the microarchitectural details of the target platforms while maintaining simulation speed. To solve that, this work proposes replacing the traditional approach of generating code annotations with all processor internal details by the use of neural networks. Training the neural network for a specific processor enables considering its internal details when estimating the cost of executing each basic block of the software in the target processor.**

I. Introduction

The development of embedded software and the analysis of its impact in the overall system performance are critical tasks in embedded system design, since most of embedded functionality is typically executed by processors. As consequence, the improvement of processors and design tools capable of helping embedded designers to deal with the corresponding challenges are an important research area. As an example, current large interest of system design community worldwide in RISC-V processors demonstrates that issue [1]. Among them, embedded software simulation plays an important role in embedded design flow, as an enabler capable of evaluating both the functionality and performance of the software early in the design process.

Traditional embedded SW (software) simulators use different technologies offering solutions with different accuracy versus simulation-speed products [2]. RTL (Register Transfer Level) simulation using a HDL (Hardware Description Language) is very accurate but very slow. Just displaying the Linux login screen takes more than 30 minutes using Synopsys VCS on an Intel Core i7 870 at 2.93GHz [3]. Although, speed can be multiplied by 3-4 times if the RTL code is translated to C, simulation speed is still very low to simulate embedded SW [4]. ISS (Instruction-Set Simulators) increase simulation speed by abstracting the underlying HW (hardware) and executing the cross-compiled target instructions on the host. Two technologies can be differentiated: interpretation and virtualization.

Interpretation takes each instruction and models its execution on the target. Depending on the modeling detail, it can provide a variety of accuracy vs. speed trade-offs. Gem5 models the CPU pipeline and memory architecture, thus being able to provide accurate cycle estimations [5].

Virtualization provides an execution interface in which the target binary is executed on the host using "Just-In-Time Code Morphing" (binary translation). That is, each target instruction is translated to the equivalent instruction(s) in the host, thus providing instruction accuracy. This is the dominant technology in commercial SW simulators like OVP [6]. These simulators provide interfaces to the model of the target HW platform, which can be modeled in an RTL, for high accuracy, or SystemC, for speed. Part of the HW model can be the cache and the rest of the memory hierarchy, the bus, peripherals, DMA, etc. Such a platform, although significantly slower than the very fast ISS simulator alone, significantly improves the timing estimation accuracy. VPSim is an example of such a platform, based on QEMU and SystemC [7]. QEMU, is a popular open-source virtualization tool [8]. It has been extended to support natively cache modeling, thus improving accuracy and speed w.r.t. an equivalent SystemC model of the memory hierarchy [9].

Native or host-compiled simulation can provide similar accuracy to commercial, state-of-the-art virtualization technologies at a much higher simulation speed. The main technique used by host-compiled simulation is to back-annotate the application code with the performance information from the target binary (or assembler) and then, to compile and execute the code in the host. The annotated application code ensures functional accuracy while the annotation code provides the actual execution characteristics of the target. Depending on the accuracy required, the annotation code can be larger or smaller. Typically, the back-annotation process requires identifying the software BBs (Basic Blocks) and estimating the cost of each block by analyzing the corresponding assembler [10]. A BB is the smallest sequence of instructions between two consecutive jumps, branches, or arrival/return points from them (i.e. the jumps and branches shall be the last instruction of a basic block, and an arrival/return point the first one). Currently, host-compiled simulation is a mature simulation and performance analysis technology with more than 10 years of research and development [11].

Host-compiled simulation does not require generating fully executable target binaries, so the effort of performing large portings for initial evaluations is avoided. However, for the generation of an accurate simulation model, information about the ISA and the micro-architecture of the target processors

are still needed. The difficulty of this technique is that, first, obtaining information of all the microarchitectural details and timing effects of a processor is difficult, since most times this is not provided by vendors. Secondly, including these effects on the modeling tools typically requires manual tool adaptation, preventing the adoption of this approach by most companies.

To solve this, we explore the use of AI (Artificial Intelligence) as a solution to create generic host-compiled simulation tools providing high accuracy for any platform without requiring specific tool adaptation from the user. The idea is that the AI can be automatically trained for a specific target processor, extracting the micro-architectural details from a real board or a slower simulation model, as RTL. That way, the resulting simulation tool is able to estimate the number of clock cycles required by a certain target processor to execute each BB in the assembler code of an application software, and feed the back-annotation with this estimation without any specific knowledge or manual effort. This would avoid complex architectural-dependent algorithms and models, as those proposed in the literature.

In this paper, we use as embedded target processor both a simulated RISC-V model running in Gem5 [12], and then we validate the technique with a real HW RISC-V platform [13]. Although most of the RISC-V implementations are open source, and therefore it eases the modeling effort a little bit, in this work we use it as an example target processor that will allow us to compare better with other state-of-the-art simulation techniques that require better knowledge of the processor internal architecture. Our new host-compiled simulation technique can be applied to whatever processor, even if its internal architecture is unknown. It only needs access to the real processor or high-accurate simulation model (which may be blinded) for the profiling of a benchmark. Then, the ML model, in our case a NN (Neural Network), will learn and model the target processor, and this ML model will be available for as many simulations and application codes are needed.

The structure of the paper is the following: in the next section, the state-of-the-art is reviewed and our work placed in context; in the following one, the methodology for the AI-based performance annotation is explained; the Results section is divided into the test and experiments carried out using the Gem5 simulator as target, and the validation on the real HW processor; finally, in the last section, conclusions are derived and future work is proposed.

II. Related Work

Host-compiled simulation is, in general, a powerful approach to provide virtual prototyping solutions combining high simulation speed with reasonably accurate evaluations of target platform parameters, such as performance or power consumption. Source codes are annotated to obtain these estimations and automatically inject them in the simulation, resulting in timed prototypes where architecture exploration and early software developments are easier to perform than in the real platforms since no code porting is required. Traditional host-compiled simulation is performed by analyzing the source code to identify its basic blocks, cross-compiling and finding in the target assembler or binary code the piece of code that corresponds to each source code block [14].

However, this task has several challenges. The first one focuses on correlating the source and assembler codes, especially when compiler optimizations are applied. In that context, several approaches optimizing the mapping of both control-flows have been proposed, as [15], [16]. However, they cannot guarantee that a complete matching will be found.

Alternatively, the use of IR (Intermediate Representation) code has also been proposed, since at this level most compiler optimizations are already solved and the matching can be done more easily [14], [17]. However, the use of assembler code in systems headers or back-end mappings is still a challenge.

The second one is how to obtain the clock cycles corresponding to the assembler instructions in a block. The easiest way to do this is to assume a constant factor: one cycle per instructions, as some ISSs do [18], or the CPI factor (Cycles Per Instruction) if available. However, this approach is not very accurate. To resolve this, the use of a constant value per assembler instruction or even solutions to solve micro-architecture elements, such as out-of-order execution [19], or to model multi-level memory hierarchies [20], [21], has been applied to improve estimations. Another element analyzed in the literature is how to minimize the number of annotations to speed-up simulations [22].

The alternative proposed in this paper for this second problem, focuses on the use of ML to solve the problem. ML has proven its utility in many areas of the Electronic Design Automation (EDA). However, its use on software simulation and performance analysis is limited [23], [24]. In [25], the C++ source code is analyzed based on its functional features of complexity, conditional branches, memory accesses, etc. This data feeds a Neural Network (NN) that predicts execution time and energy consumption, trained on a set of benchmarks on the target platform. At a much lower level of abstraction, [26] proposes ML for RTL power modeling of CPUs. Nevertheless, to the best of our knowledge, it has not yet been used to predict the number of cycles a BB will take, as required to preform host-compiled simulation.

III. IA-based performance annotation/methodology

In host-compiled simulation, the application code is instrumented with extra code capable of modeling the performance figures that can be obtained from executing the original application code on the target platform. This extra code is called the annotation code.

Traditionally, these performance figures are extracted and added per basic block in the application code, as all instructions within each basic block are always executed together. That way, the annotation code models the estimated time $\hat{Tt}^i$ of the execution of B_i in the target processor. Although in reality, application and annotation codes are intermixed, the

simulation time of each block B_i can always be computed as: $Ht^i = Ht^i_{an} + Ht^i_{app}$, where Ht^i_{app} is the execution time in the host machine taken by the original application code (without any annotation) and Ht^i_{an} is the additional execution time (in the host) introduced by the annotation code. By considering all the blocks in a simulation experiment, the total simulation time Ht is obtained: $Ht = Ht_{an} + Ht_{app}$.

Note that if only the execution time of the instructions in a BB is estimated, the annotation code can be as simple as $\hat{Tt} \mathrel{+}= \hat{Tt}^i$. Just by estimating a mean number of clock CPI (Cycles Per Instruction), both the accuracy and the speed are better than by virtualization. However, apart from the number of cycles required by the execution of the instructions, an accurate execution time estimation requires modeling other micro-architectural details such as jump or data dependencies. This requires a more complex, processor-specific annotation code and, therefore, a greater simulation time.

The new approach presented in this paper is the use of a NN to predict the number of cycles of each BB, instead of adding the cost of the individual assembler instructions and modeling processor internals in the annotation code. To predict this number, the NN receives the assembler instructions conforming each BB as input. The assembler instructions are 32 bit numbers, however, considering them as integer inputs is not adequate since the meaning of these bits is purely symbolic and arbitrary. As a result, each instruction identifier (48 for the RV64I ISA) has been encoded as one-hot. Additionally, since BBs have a variable number of instructions, our proposal is to fix a maximum number of instructions per block, splitting larger blocks. BB with less instructions than the maximum are filled with an entry value as "no instruction", which is represented in the one-hot encoding as all zeros in order to do not trigger any activation. Additionally, to cover effects such as processor pipeline dependencies, this maximum number of instructions per BB is larger than the number of stages in the processor pipe, and the network also includes inputs for inter-BB dependencies. The dependency inputs are generated by analyzing if there are RAW (Read After Write) dependencies, which may have a high impact in ordered processors. The dependency of the first instruction of a BB is not calculated (and therefore it is always zero) in order to make the BBs independent.

After that, a state-of-the-art host-compiled simulation tool [27] was modified to back-annotate the basic blocks with the numbers obtained by the network instead of performing a traditional cycle count.

A. Benchmark generation

With the intention of exploring the capacity of neural networks to predict the cycle cost of a set of assembly instructions, we need a dataset capable of exploring the instructions in the ISA and their corresponding cycle costs. We have chosen to create a synthetic dataset of BBs of random instructions accessing random registers and/or memory addresses, with certain restrictions such as that jump instructions are only allowed as the last instruction of a BB. The maximum number of instructions per BB is decided to be slightly larger than the pipe depth. Therefore, it is 6 for the real HW ESP32C3 processor [13], and 8 for the O3CPU simulated in Gem5 [12]. This synthetic dataset allows us fast coverage of all possible situations that may happen in the processor.

In order to make the BBs independent from each other, the program cleans the processor pipeline by forcing it to execute 10 NOP instructions before each valid BB. This guarantees that the processor pipeline is free of dependencies and that the buses are free before each BB. If the BB to estimate is smaller than the maximum BB length, the missing instructions are also filled with NOPs as they spend exactly one cycle, and therefore eases the automation of the dataset calculation. This approach enabled us to evaluate approximately 200,000 BBs, which were used to train the neural network. To balance the generated dataset, the BBs containing less instructions were repeated when they exhausted all their possible combinations to compensate for the BBs containing more instructions (and therefore a larger number of executed combinations). Since detecting cache misses is out of the scope of our technique, all BBs that have triggered a cache miss have been excluded from the dataset.

B. *The* intrinsics *problem*

In our technique, as well as in most host-compiled techniques, the BB division and its annotation is introduced at source code (e.g. C/C++) or at IR (as in our case by using LLVM). However, compilers can detect certain code patterns and automatically replace them by optimized *intrinsic* assembler code or function calls. This causes a discrepancy between the annotated BBs, and the actual BBs present in the final assembler or executable code. So, the code parts affected by the *intrinsics* cannot be properly annotated.

To solve this problem, we have provided the compiler with a C implementation of these *intrinsics*. Currently the list is not complete, and they may not be as optimal as the original ones, nevertheless, they allow us to properly annotate all the code of the tested application examples and provide an estimate for the final optimized version.

Division and floating point operations represent a similar problem. If the target processor has floating point unit, it can execute them as a single instruction, however when the processor does not have floating point unit, these instructions are executed as a call to a routine or inserting extra assembler code that may depend on the operands. For this reason, in this first proof of concept of our technique, these operations have been avoided and only a RISC-V base integer ISA (I) has been simulated.

C. NN model description

The NN used to estimate the BBs execution cycles is a 1D (Dimensional) CNN (Convolutional Neural Network). It is divided in two parts: the 1D-CNN input block, that shares the weights used to compute each instruction/operation of the BB; and the FC (Fully Connected) header block that produces

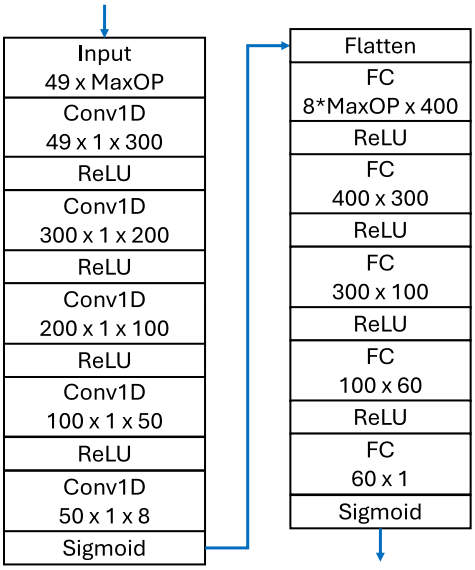

Fig. 1. NN architecture.

the regression output to get the estimated execution cycles. Figure 1 shows the detailed architecture of the NN.

IV. Results

In order to get the target number of execution cycles, first we have performed high-accurate but slow simulations in Gem5, where as an advantage, we can more easily change the processor architecture. Gem5 provides very accurate execution cycles estimates that we use as targets for the ML training and testing. Then, in order to validate the accuracy of our technique in real HW, we have also trained and tested our ML model in the RISC-V processor of the ESP32C3 microcontroller.

In order to provide results from real examples, a detailed analysis of the NN results was conducted on widely used benchmarks: bubble-sort, SHA-256, Dijkstra's algorithm, Fibonacci function, and the Mersenne twister (random number generator).

A. Gem5/high-accurate simulation based tests

The Gem5 RISC-V processor O3CPU, which is a 64-bit processor, with only the integer base (I) ISA. It has six processing stages. Although this processor is virtual, it simulates the behavior of more complex units, even executing instructions out of order. For a first proof of concept, we have restricted the processor so that it could only fetch one instruction at a time, limiting each of its stages to a single instruction. This allowed us to simplify the model for parameterization. Additionally, cache modifications were made to minimize its impact during execution.

Table I shows the errors between the Gem5 simulation (considered as the true target) and the execution cycles estimated by our simulator for the full benchmark functions. Moreover, Table II shows the average errors of the BBs without taking into account the number of times each BB is executed.

TABLE I
Function APE (Absolute Percentage Error)

	bubble	SHA-256	Dijkstra's	Fibonacci	M. twister
1C	28.97%	29.71%	40.81%	32.52%	33.59%
CPI	22.16%	20.90%	1.81%	16.07%	14.23%
C(OP)	10.85%	15.18%	3.11%	5.64%	6.30%
NN	9.67%	9.04%	5.47%	2.96%	1.04%

TABLE II
BBs MAPE

	bubble	SHA-256	Dijkstra's	Fibonacci	M. twister
1C	35.68%	33.21%	34.79%	26.69%	31.63%
CPI	12.96%	21.31%	13.18%	36.62%	20.14%
C(OP)	9.22%	18.52%	13.95%	24.40%	13.81%
NN	8.92%	12.42%	8.79%	13.78%	13.24%

As can be noticed in Table I, the CPI technique gets an outlier for the Dijkstra's algorithm function. In order to understand this, figures 2, 3, and 4 shows the signed error of each BB against its number of repetitions (i.e. the number of times that each BB is executed) for the Dijkstra's algorithm function. We can observe that there is a BB which is repeated an order of magnitude more times than the other BBs. This particular BB is composed of a single *JAL* (Jump And Link) operation, which is an operation with a very variable duration depending on the state of the pipe. Using our synthetic dataset to calculate all the costs per operation (*C(OP)*s), the *JAL* operation got 1.5 cycles on average. However, surprisingly, for the Dijkstras's algorithm function, the BB containing the single *JAL* got an average of 1.74 cycles, which is by pure coincidence very close to the CPI for every operation of the ISA, which is 1,72 cycles.

The differences between Table I and Table II are caused by the fact that not all the BBs are executed evenly. Some of them are not even executed and others are executed several orders of magnitude more times than the rest. Additionally, the actual signed errors of each BB obtained during the execution of the function can partially cancel each other (i.e., overestimations compensate underestimations), since the *absolute* in the function APE (Absolute Percentage Error) is applied to the final total error.

B. Real HW validation

In order to validate the accuracy of our technique in real HW, we have also trained and tested our ML model in the RISC-V processor of the ESP32C3 microcontroller [13]. It is a 32-bit with an integer base architecture (I) that includes multiplication/division capabilities (M) and compressed standard extensions (C), and has four stages in the pipeline. So, we decided to limit the number of instructions per BB to six.

This processor incorporates two timers, of which timer 0 was selected for the purpose of this study. It operates at a clock speed of 160 MHz, while the timer used operates at 80 MHz. To accurately measure the number of cycles required

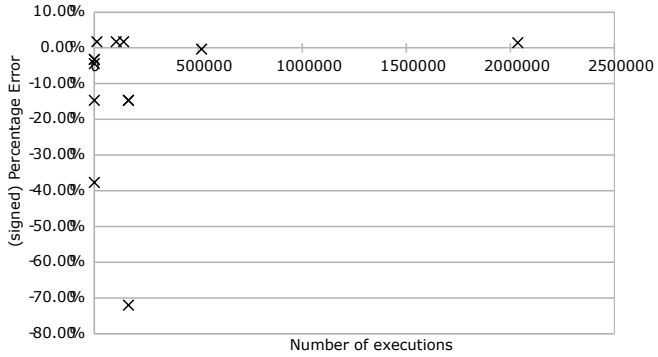

Fig. 2. CPI BBs of the Dijkstra's algorithm.

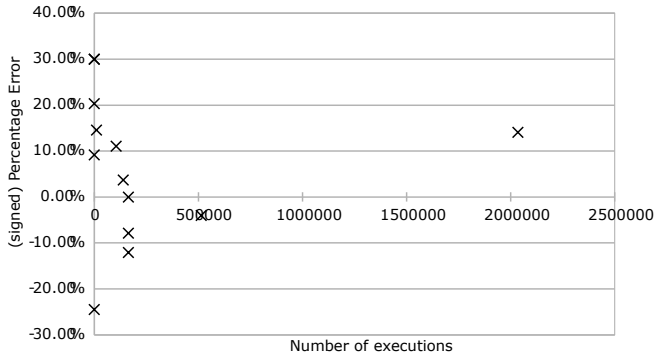

Fig. 3. C(OP) BBs of the Dijkstra's algorithm.

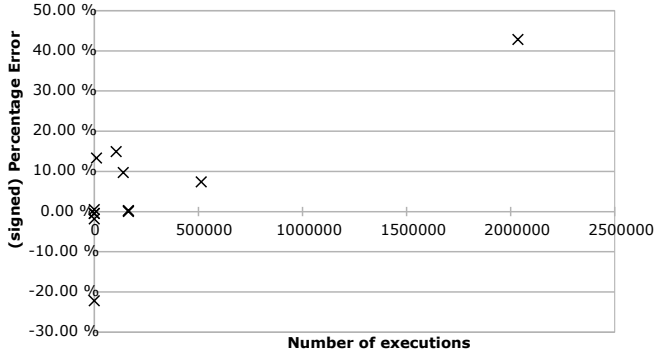

Fig. 4. NN BBs of the Dijkstra's algorithm.

for a BB with a single instruction, the minimum prescaling available on this processor, which is two, was chosen, allowing a measurement with approximately four processor cycles per clock cycle (of the timer).

Due to the lack of clock precision and the fact that when executing the same group of instructions, the processor should use the same number of cycles, in order to obtain accurate data, the test group was executed multiple times, generating a sample of n repetitions of the test to be measured. In each iteration, the test was repeated four times consecutively: once, then twice, then three times, and finally four times. The objective was to measure the number of cycles required for each test. However, owing to the lack of sufficient precision, the obtained data resulted in a stepped graph instead of a straight line. But with our technique, we have improved the precision sufficiently.

TABLE III
REAL HW BUBBLE SORT (SIGNED) PERCENTAGE ERRORS OF EACH BB AND TOTAL (LAST ROW)

C(OP)	NN	C(OP)	NN
-14.3%	2.7%	-8.5%	-5.9%
12.5%	42.8%	0.0%	0.9%
-11.1%	-0.4%	-14.8%	-10.2%
-10.3%	-4.6%	0.0%	8.2%
-5.6%	13.6%	-8.0%	-9.6%
12.5%	42.8%	-14.3%	0.4%
0.0%	16.2%	-5.6%	8.8%
-11.1%	-5.0%	10.0%	12.3%
	Total:	-8.2%	-2.0%

To address this challenge, it was decided to increase the number of tests generated and perform a linear regression of the tests and measurements obtained. The linear regression returned a function of the form ax + b = y, where 'a' is the value of interest, since it represents the number of cycles required for the execution of the test when multiplied by four, due to the difference in speeds between the timer and the processor. This adjustment is necessary to understand how many cycles the processor requires for a particular test.

To ensure the highest possible accuracy, the decision was made to double the number of tests performed again, carrying out a total of 24 tests. This was done to obtain more robust data and reduce any variability in measurements. In addition, we sought to find the trend of the data by graphing them, so as to estimate the number of cycles required for each test. Initially, a spreadsheet was used to analyze the data, but in order to automate the process, we opted to use Python for data processing. This allowed the data to be read directly from the serial console, making it easier to obtain and analyze the results.

Table III presents two result columns: one corresponding to the reference simulator, which uses the *C(OP)* technique, and the corresponding to our NN. Furthermore, at the bottom of the table, the totals are aggregated. The actual cycle numbers for the benchmark could not be provided due to the possibility of cache faults in the machine used in the tests with our measurement technique. However, in this case, by using sufficiently small basic blocks and repeating the test multiple times, cache faults are avoided. As can be seen, despite the absolute maximum error of the NN (42.8%) being larger than the maximum error using the *C(OP)* annotation technique (14.8%), the accumulated error is smaller when using the NN (-2%) than when using the *C(OP)* (-8,2%).

V. CONCLUSIONS AND FUTURE WORK

In this paper we have proved that a NN can give accurate execution time estimations of a given target microprocessor. Since the scope of our technique is the pipe effects, it is better

suited to be combined with a model or simulator that takes into account the caches and memory hierarchy. Besides, the process of training a new NN for a new processor, since it involves the execution of the benchmark in the real HW or an accurate but slow simulator, it can take up to several days. Therefore, our technique excels in application domains where the processor is fixed but many SW simulations need to be performed, like drones or system platform design where there are only a few fixed processors but the caches, memory hierarchy, and other peripherals need to be explored.

As this paper presets a first proof of concept, we have selected a relatively simple target processor. However, this simplicity minimizes the improvement achieved with respect to other annotation techniques, as in this context they can get decent accuracy. It is expected that, for a more complex out-of-order processor, the alternative annotation techniques, based on linear relationships between the (number of) instructions, will have poor accuracy whereas the NN should be able to learn the non-linearities of such a complex processor.

Since the best strength of this simulation technique is to avoid the target processor modeling efforts, it becomes even more powerful when the target is a vendor processor from which some architecture details are not available. Therefore, the next future work will be to test this technique to predict execution times on a more complex processor in which the *C(OP)* annotation technique will not be accurate enough, and which architecture details are not available. Then, the real HW processor will be needed or a protected (black box) accurate (but slow) simulator. Moreover, the ML model can also be trained to predict other figures, as power consumption.

ACKNOWLEDGMENT

The authors would like to thank TEKNE for letting us to use their use case to validate our tool. This work has been partially funded by the EU and the Spanish Mineco through the ECSEL Joint Undertaking (JU) under grant agreement N. 101007350 and the PID2020-116417RB-C43, TALENT project.

REFERENCES

[1] K. Leung, "The growing momentum of risc-v in europe," https://riscv.org/blog/2023/07/the-growing-momentum-of-risc-v-in-europe/, 2022.

[2] S. Ha and J. Teich, *Handbook of hardware/software codesign*. Springer Publishing Company, Incorporated, 2017.

[3] J. Miura, H. Miyazaki, and K. Kise, "A portable and linux capable risc-v computer system in verilog hdl," *arXiv preprint:2002.03576v2*, 2020.

[4] M. Abderehman, J. Patidar, J. Oza, Y. Nigam, T. A. Khader, and C. Karfa, "FastSim: A fast simulation framework for high-level synthesis," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 5, pp. 1371–1385, May 2022.

[5] J. Lowe-Power, A. M. Ahmad, A. Akram, M. Alian, R. Amslinger, M. Andreozzi, and et al., "The gem5 simulator: Version 20.0+," *arXiv preprint arXiv:2007.03152*, 2020.

[6] "Open Virtual Platforms," https://ovpworld.org.

[7] F. Jebali, O. Matoussi, A. Wicaksana, A. Charif, and L. Zaourar, "Decoupling processor and memory hierarchy simulators for efficient design space exploration," in *Proceedings of the DroneSE and RAPIDO: System Engineering for constrained embedded systems*. Association for Computing Machinery, 2022, pp. 47–52.

[8] "QEMU: A generic and open source machine emulator and virtualizer," https://qemu.org.

[9] M. Badaroux, J. Dumas, and F. Pétrot, "Fast instruction cache simulation is trickier than you think," in *Proceedings of the DroneSE and RAPIDO: System Engineering for Constrained Embedded Systems*. Association for Computing Machinery, 2023, p. 48–53.

[10] J. Schnerr, O. Bringmann, A. Viehl, and W. Rosenstiel, "High-performance timing simulation of embedded software," in *Proceedings of the 45th Annual Design Automation Conference*, ser. DAC '08. Association for Computing Machinery, 2008, p. 290–295.

[11] O. Bringmann, W. Ecker, A. Gerstlauer, A. Goyal, D. Mueller-Gritschneder, P. Sasidharan, and S. Singh, "The next generation of virtual prototyping: Ultra-fast yet accurate simulation of HW/SW systems," in *Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2015, pp. 1698–1707.

[12] "O3CPU: Out of order cpu model," https://www.gem5.org/documentation/general_docs/cpu_models/O3CPU.

[13] "ESP32C3: A cost-effective risc-v mcu with wi-fi and bluetooth 5 (le) connectivity for secure iot applications," https://www.espressif.com/en/products/socs/esp32-c3.

[14] S. Chakravarty, Z. Zhao, and A. Gerstlauer, "Automated, retargetable back-annotation for host compiled performance and power modeling," in *International Conference on Hardware/Software Codesign and System Synthesis (CODES+ISSS)*, 2013.

[15] Z. Wang and J. Henkel, "Accurate source-level simulation of embedded software with respect to compiler optimizations," in *Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2012, pp. 382–387.

[16] S. Stattelmann, O. Bringmann, and W. Rosenstiel, "Fast and accurate source-level simulation of software timing considering complex code optimizations," in *Proceedings of the 48th Design Automation Conference*, ser. DAC '11. Association for Computing Machinery, 2011, p. 486–491.

[17] O. Matoussi and F. Pétrot, "A mapping approach between IR and binary CFGs dealing with aggressive compiler optimizations for performance estimation," in *23rd Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2018, pp. 452–457.

[18] B. Bailey, "System level virtual prototyping becomes a reality with OVP donation from imperas," *White Paper*, June 2008.

[19] R. Plyaskin and A. Herkersdorf, "Context-aware compiled simulation of out-of-order processor behavior based on atomic traces," in *IEEE/IFIP 19th International Conference on VLSI and System-on-Chip*, 2011, pp. 386–391.

[20] K. Lu, D. Müller-Gritschneder, and U. Schlichtmann, "Fast cache simulation for host-compiled simulation of embedded software," in *Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2013, pp. 637–642.

[21] O. Matoussi and F. Pétrot, "Modeling instruction cache and instruction buffer for performance estimation of VLIW architectures using native simulation," in *Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2017, pp. 266–269.

[22] J. Benz, C. Gerum, and O. Bringmann, "Advancing source-level timing simulation using loop acceleration," in *Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2018, pp. 1393–1398.

[23] G. Huang, J. Hu, Y. He, J. Liu, M. Ma, Z. Shen, and et al., "Machine learning for electronic design automation: A survey," *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, vol. 26, no. 5, June 2021.

[24] E. S. Alcorta Lozano, A. Gerstlauer, C. Deng, Q. Sun, Z. Zhang, C. Xu, and et al., "Special session: Machine learning for embedded system design," in *Proceedings of the International Conference on Hardware/Software Codesign and System Synthesis*, ser. CODES/ISSS '23 Companion. Association for Computing Machinery, 2023, p. 28–37.

[25] V. Muttillo, P. Giammatteo, and V. Stoico, "Statement-level timing estimation for embedded system design using machine learning techniques," in *Proceedings of the ACM/SPEC International Conference on Performance Engineering (ICPE)*. Association for Computing Machinery, 2021, p. 257–264.

[26] A. K. A. Kumar, S. Al-Salamin, H. Amrouch, and A. Gerstlauer, "Machine learning-based microarchitecture-level power modeling of CPUs," *IEEE Transactions on Computers*, vol. 72, no. 4, pp. 941–956, April 2023.

[27] H. Posadas, E. Villar, and M. Sanchez-Renedo, "Accelerating host-compiled simulation by modifying IR code: Industrial application in the spatial domain," in *XXXIV Conference on Design of Circuits and Integrated Systems (DCIS)*, 2019, pp. 1–6.